

LeetCode Exercise:

1. Big Countries

Problem Summary

You are given a table World with the following columns:

- name (country name)
- continent
- area (in square km)
- population
- GDP
- A country is considered “big” if:
 - Its area $\geq 3,000,000$, OR
 - Its population $\geq 25,000,000$

Task

Write an SQL query to return the name, population, and area of all big countries.

</> Code

MySQL Auto

```
1
2 SELECT name, population, area
3 FROM World
4 WHERE area >= 3000000 OR population >= 25000000;
```

Saved

☒ Testcase

☒ Test Result

Accepted Runtime: 69 ms

☒ Case 1

Input

World =

name	continent	area	population	gdp
Afghanistan	Asia	652230	25500100	20343000000
Albania	Europe	28748	2831741	12960000000
Algeria	Africa	2381741	37100000	188681000000
Andorra	Europe	468	78115	3712000000
Angola	Africa	1246700	20609294	100990000000

Output

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

Expected

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

2.Find Customer Referee

Problem Summary

You are given a table Customer with the following columns:

- id (customer ID)
- referee_id (ID of the referring customer)
- name (customer name)

Task

Find the names of customers who were NOT referred by the customer with id = 2.

- Note: Customers with NULL in referee_id should also be included.

Code

MySQL Auto

1

SELECT name

2

FROM Customer

3

WHERE referee_id != 2 OR referee_id IS NULL;

Saved

Testcase

Test Result

Accepted

Runtime: 314 ms

Case 1

Input

Customer =

id	name	referee_id
1	Will	null
2	Jane	null
3	Alex	2
4	Bill	null
5	Zack	1
6	Mark	2

Output

name
Will
Jane
Bill
Zack

Expected

name
Will
Jane
Bill
Zack

3. Not Boring Movies

Problem Summary

You are given a table Cinema with the following columns:

- id (movie ID)
- description (movie type)
- movie (movie name)
- rating (movie rating)

Task

Find all movies that are:

- Not boring $\rightarrow$ description $\neq$ 'boring'
- Have odd-numbered IDs $\rightarrow$ id $\% 2 = 1$

Return the result sorted by rating in descending order.

</> Code

MySQL Auto

```
1 SELECT *
2 FROM Cinema
3 WHERE description != 'boring'
4 AND id % 2 = 1
5 ORDER BY rating DESC;
```

Saved

☒ Testcase

☒ Test Result

Accepted Runtime: 67 ms

☒ Case 1

Input

cinema =

id	movie	description	rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	irish	boring	6.2
4	Ice song	Fantasy	8.6
5	House card	Interesting	9.1

Output

id	movie	description	rating
5	House card	Interesting	9.1
1	War	great 3D	8.9

Expected

id	movie	description	rating
5	House card	Interesting	9.1
1	War	great 3D	8.9

4.Invalid Tweets

Problem Summary

You are given a table Tweets with the following column:

- tweet_id (ID of the tweet)
- content (text of the tweet)

Task

Find the IDs of tweets that are invalid, where a tweet is considered invalid if:

- The length of the content is greater than 15 characters

 Code

MySQL   Auto

```
1 SELECT tweet_id
2 FROM Tweets
3 WHERE LENGTH(content) > 15;
```

Saved

☒ Testcase |  Test Result

Accepted Runtime: 69 ms

☒ Case 1

Input

Tweets =

tweet_id	content
1	Let us Code
2	More than fifteen chars are here!

Output

tweet_id
2

Expected

tweet_id
2

5. Recyclable and Low-Fat Products

Problem Summary

You are given a table Products with the following columns:

- product_id
- low_fats (values: 'Y' or 'N')
- recyclable (values: 'Y' or 'N')

Task

Find the product IDs of products that are:

- Low fat (low_fats = 'Y')
- Recyclable (recyclable = 'Y')

</> Code

MySQL Auto

```
1 SELECT product_id
2 FROM Products
3 WHERE low_fats = 'Y'
4 AND recyclable = 'Y';
```

Saved

Testcase | Test Result

Accepted Runtime: 72 ms

Case 1

Input

Products =

product_id	low_fats	recyclable
0	Y	N
1	Y	Y
2	N	Y
3	Y	Y
4	N	N

Output

product_id
1
3

Expected

product_id
1
3

6.Article Views I

Problem Summary

You are given a table Views with the following columns:

- article_id
- viewer_id
- author_id
- view_date

Task

Find all authors who viewed at least one of their own articles.

- Return the result as a list of unique author IDs
- Sort the result in ascending order.

```
1 select distinct author_id as id from Views
2 where author_id = viewer_id
3 order by id;
```

Saved

✓ Testcase | >_ Test Result

Accepted Runtime: 69 ms

✓ Case 1

Input

Views =

article_id	author_id	viewer_id	view_date
1	3	5	2019-08-01
1	3	6	2019-08-02
2	7	7	2019-08-01
2	7	6	2019-08-02
4	7	1	2019-07-22
3	4	4	2019-07-21

⌵ View more

Output

id
4
7

Expected

id
4
7